Question 1:

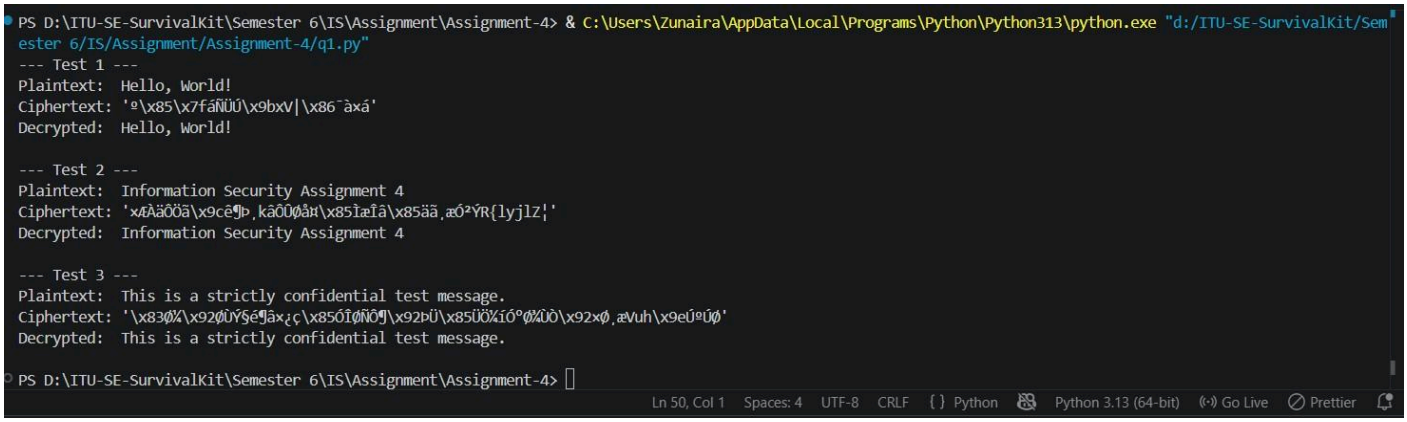
Source Code (q1.py)

```
def pad(text, block_size=8):
    """Pads text to ensure it's a multiple of the block size."""
    padding_len = block_size - (len(text) % block_size)
    return text + chr(padding_len) * padding_len


def unpad(text):
    """Removes padding."""
    padding_len = ord(text[-1])
    return text[:-padding_len]


def custom_encrypt(plaintext, key):
    if len(key) < 8:
        raise ValueError("Key must be at least 64 bits (8 characters
long).")

    plaintext = pad(plaintext)

    # Step 1: Substitution (Add key character value modulo 256)
    substituted = []
    for i, char in enumerate(plaintext):
        key_char = key[i % len(key)]
        sub_char = chr((ord(char) + ord(key_char)) % 256)
        substituted.append(sub_char)

    # Step 2: Transposition (Reverse every 8-character block)
    ciphertext = ""
    for i in range(0, len(substituted), 8):
        block = substituted[i : i + 8]
        ciphertext += "".join(block[::-1])
```

```
    return ciphertext


def custom_decrypt(ciphertext, key):
    # Step 1: Reverse Transposition (Reverse every 8-character block
back)
    untransposed = []
    for i in range(0, len(ciphertext), 8):
        block = ciphertext[i : i + 8]
        untransposed.extend(list(block[::-1]))

    # Step 2: Reverse Substitution (Subtract key character value
modulo 256)
    plaintext = ""
    for i, char in enumerate(untransposed):
        key_char = key[i % len(key)]
        orig_char = chr((ord(char) - ord(key_char)) % 256)
        plaintext += orig_char

    return unpad(plaintext)


# --- Test Cases ---
if __name__ == "__main__":
    key = "SuperSecretKey"  # > 64 bits
    messages = [
        "Hello, World!",
        "Information Security Assignment 4",
        "This is a strictly confidential test message.",
    ]

    for i, msg in enumerate(messages):
        cipher = custom_encrypt(msg, key)
        decrypted = custom_decrypt(cipher, key)
        print(f"--- Test {i+1} ---")
        print(f"Plaintext:  {msg}")
        print(f"Ciphertext: {repr(cipher)}")
```

```
        print(f"Decrypted:   {decrypted}\n")
```

Screenshot



Report Explanation: Weakness & Improvements

- **Weakness:** The primary weakness of this algorithm is its simple modular addition for substitution. Because the key simply repeats, it acts much like a Vigenère cipher applied at the byte level. This makes it highly susceptible to known-plaintext attacks (KPA) and frequency analysis if a large amount of ciphertext is collected. The transposition step is also static (reversing 8 bytes), which doesn't provide true diffusion.
- **Improvement:** To improve this, we could replace the simple addition with a non-linear substitution box (S-box) and introduce dynamic key scheduling (generating round keys) rather than just repeating the base key. We could also add a permutation network (like bit-shifting) to ensure single-bit changes affect the entire block (diffusion).

Question 2:

Source Code (q2.py)

```
import string



# Standard English letter frequencies

ENGLISH_FREQ = {

    "A": 0.0817,
```

```
  "B": 0.0150,
  "C": 0.0278,
  "D": 0.0425,
  "E": 0.1270,
  "F": 0.0223,
  "G": 0.0202,
  "H": 0.0609,
  "I": 0.0697,
  "J": 0.0015,
  "K": 0.0077,
  "L": 0.0403,
  "M": 0.0241,
  "N": 0.0675,
  "O": 0.0751,
  "P": 0.0193,
  "Q": 0.0010,
  "R": 0.0599,
  "S": 0.0633,
  "T": 0.0906,
  "U": 0.0276,
  "V": 0.0098,
  "W": 0.0236,
```

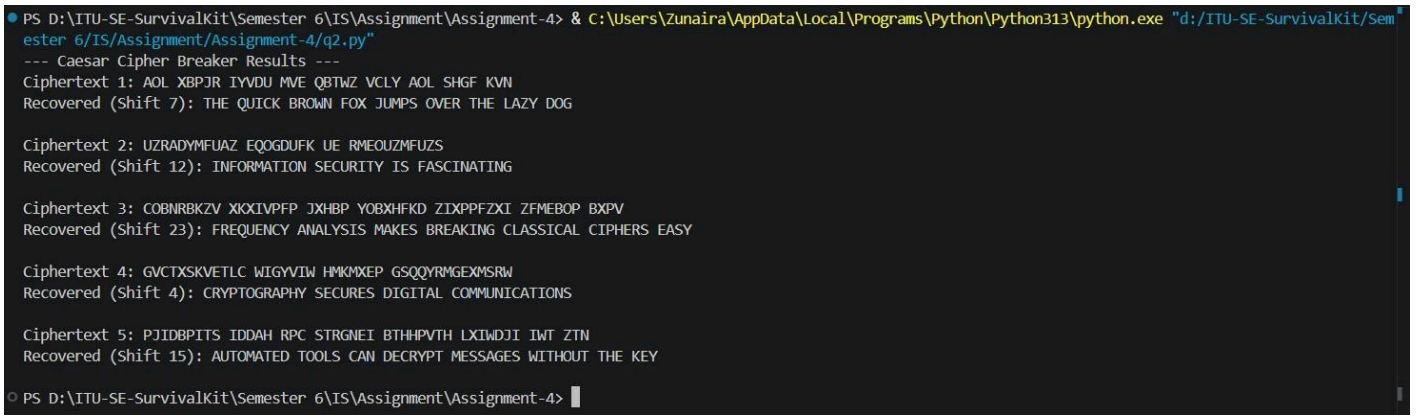
```
    "X": 0.0015,
    "Y": 0.0197,
    "Z": 0.0007,
}



def caesar_encrypt(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            base = ord("A") if char.isupper() else ord("a")
            result += chr((ord(char) - base + shift) % 26 + base)
        else:
            result += char
    return result



def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)



def score_text(text):
```

```
    """Scores text based on English letter frequencies."""
    score = 0
    total_letters = sum(1 for c in text if c.isalpha())
    if total_letters == 0:
        return 0

    text = text.upper()
    for char in string.ascii_uppercase:
        count = text.count(char)
        freq = count / total_letters
        # Chi-squared inspired scoring (lower difference is better, so we
negate it)
        score -= abs(freq - ENGLISH_FREQ[char])
    return score



def break_caesar(ciphertext):
    """Automatically breaks a Caesar cipher using frequency analysis."""
    best_score = -float("inf")
    best_shift = 0
    best_plaintext = ""
```

```
    for shift in range(26):
        plaintext = caesar_decrypt(ciphertext, shift)
        score = score_text(plaintext)
        if score > best_score:
            best_score = score
            best_shift = shift
            best_plaintext = plaintext

    return best_shift, best_plaintext



# --- Test Cases ---
if __name__ == "__main__":
    # Test your tool on at least five ciphertexts[cite: 21].
    ciphertexts = [
        caesar_encrypt("THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG", 7),
        caesar_encrypt("INFORMATION SECURITY IS FASCINATING", 12),
        caesar_encrypt("FREQUENCY ANALYSIS MAKES BREAKING CLASSICAL
CIPHERS EASY", 23),
        caesar_encrypt("CRYPTOGRAPHY SECURES DIGITAL COMMUNICATIONS", 4),
        caesar_encrypt("AUTOMATED TOOLS CAN DECRYPT MESSAGES WITHOUT THE
KEY", 15),
```

```
    ]



print("--- Caesar Cipher Breaker Results ---")

for i, ct in enumerate(ciphertexts):

    shift, plaintext = break_caesar(ct)

    print(f"Ciphertext {i+1}: {ct}")

    print(f"Recovered (Shift {shift}): {plaintext}\n")
```

Screenshot



Report Explanation: Frequency Analysis

- **How it works:** Frequency analysis works by counting the occurrences of each letter in the ciphertext. In the English language, certain letters (like E, T, A, O, I, N) appear much more frequently than others (like Z, Q, X). By mapping the highest frequency letters in the ciphertext to the known high-frequency letters in English, we can deduce the substitution mapping or the shift distance.
- **Why it succeeds:** It succeeds against ciphers like Caesar and Vigenere because these classical ciphers do not obscure the underlying statistical distribution of the plaintext. A Caesar cipher merely shifts the frequency graph to the left or right, making it trivial to realign

Question 3:

Source Code (q3.py)

```
import os

import hashlib

import json

from datetime import datetime



STATE_FILE = "file_hashes.json"

LOG_FILE = "integrity_monitor.log"




def get_file_hash(filepath):

    """Calculates the SHA-256 hash of a file."""

    hasher = hashlib.sha256()

    try:

        with open(filepath, "rb") as f:

            while chunk := f.read(8192):

                hasher.update(chunk)

        return hasher.hexdigest()

    except Exception as e:

        return None
```

```
def scan_directory(folder_path):
    """Scans the directory and returns a dictionary of file paths and
their hashes."""
    file_hashes = {}
    for root, _, files in os.walk(folder_path):
        for file in files:
            filepath = os.path.join(root, file)
            file_hash = get_file_hash(filepath)
            if file_hash:
                file_hashes[filepath] = file_hash
    return file_hashes




def log_change(message):
    """Logs changes with a timestamp."""
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    log_entry = f"[{timestamp}] {message}"
    print(log_entry)
    with open(LOG_FILE, "a") as f:
        f.write(log_entry + "\n")




def run_integrity_check(folder_path):
```

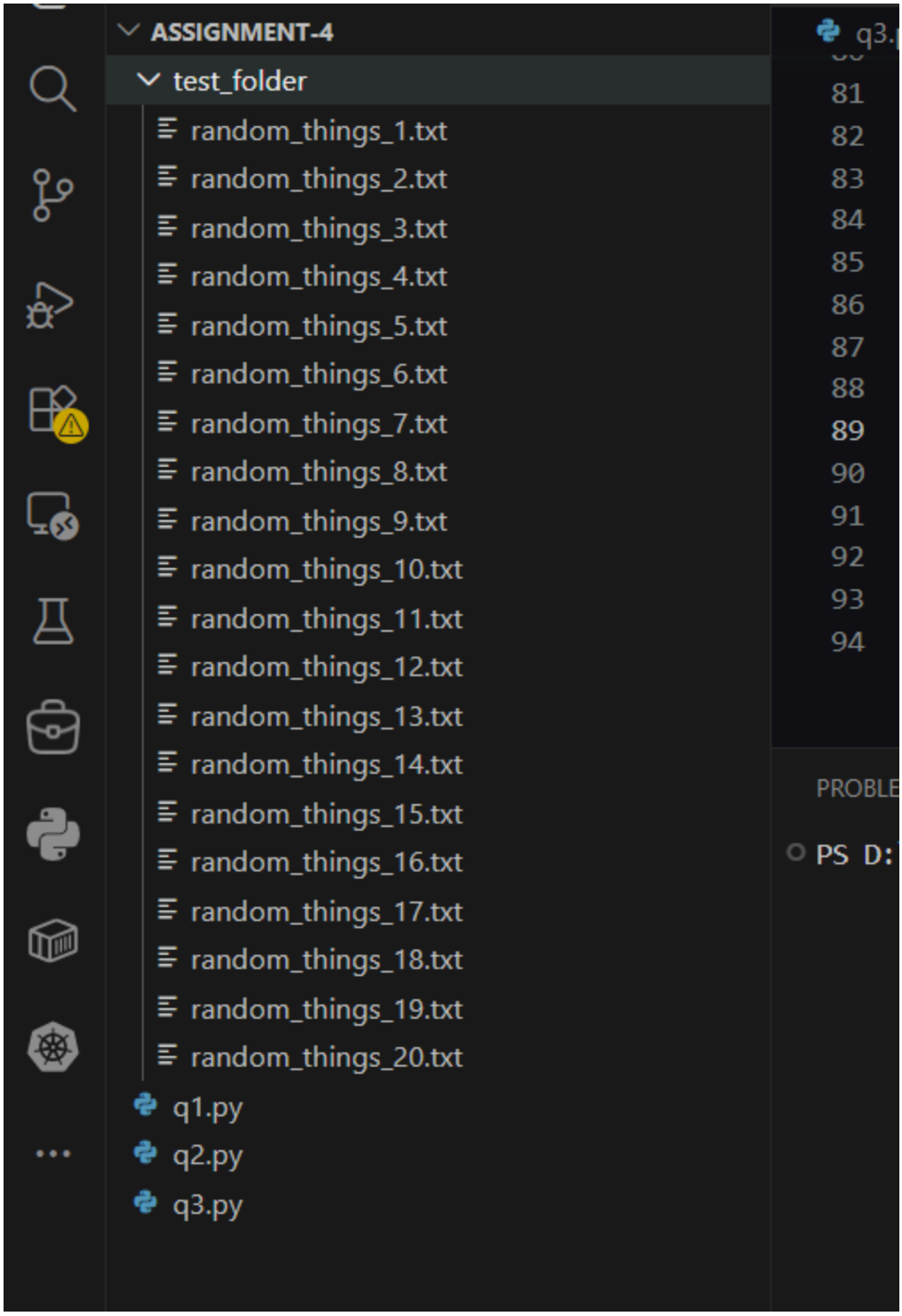
```
    print(f"Scanning folder: {folder_path}...")
    current_hashes = scan_directory(folder_path)

    # If no state file exists, this is the first run.
    if not os.path.exists(STATE_FILE):
        with open(STATE_FILE, "w") as f:
            json.dump(current_hashes, f)
        log_change("Initial scan complete. Baseline hashes saved.")
        return

    # Load stored hashes
    with open(STATE_FILE, "r") as f:
        stored_hashes = json.load(f)

    changes_detected = False

    # Check for Modified and Added files
    for filepath, current_hash in current_hashes.items():
        if filepath not in stored_hashes:
            log_change(f"ADDED: {filepath}")
            changes_detected = True
        elif stored_hashes[filepath] != current_hash:
```

```
            log_change(f"MODIFIED: {filepath}")
            changes_detected = True

    # Check for Deleted files
    for filepath in stored_hashes.keys():
        if filepath not in current_hashes:
            log_change(f"DELETED: {filepath}")
            changes_detected = True

    if not changes_detected:
        log_change("Scan complete. No changes detected.")

    # Update state file with current hashes
    with open(STATE_FILE, "w") as f:
        json.dump(current_hashes, f)


if __name__ == "__main__":
    # Target folder (Create this folder and put 20+ files in it before running)
    TARGET_DIR = "./test_folder"

    if not os.path.exists(TARGET_DIR):
```

```
        os.makedirs(TARGET_DIR)

        print(

            f"Created '{TARGET_DIR}'. Please populate it with at least 20
files and run this script again."

        )

    else:

        run_integrity_check(TARGET_DIR)
```

Screenshot

1. Create a folder named test_folder.
2. Add at least 20 random files (text files, images, etc.) into test_folder.



3. Run the script once to generate the baseline file_hashes.json file.

```
PS D:\ITU-SE-SurvivalKit\Semester 6\IS\Assignment\Assignment-4> & C:\Users\Zunaira\AppData\Loc
ester 6/IS/Assignment/Assignment-4/q3.py"
Scanning folder: ./test_folder...
[2026-05-24 14:51:44] Initial scan complete. Baseline hashes saved.
PS D:\ITU-SE-SurvivalKit\Semester 6\IS\Assignment\Assignment-4>
```

Modify a file:

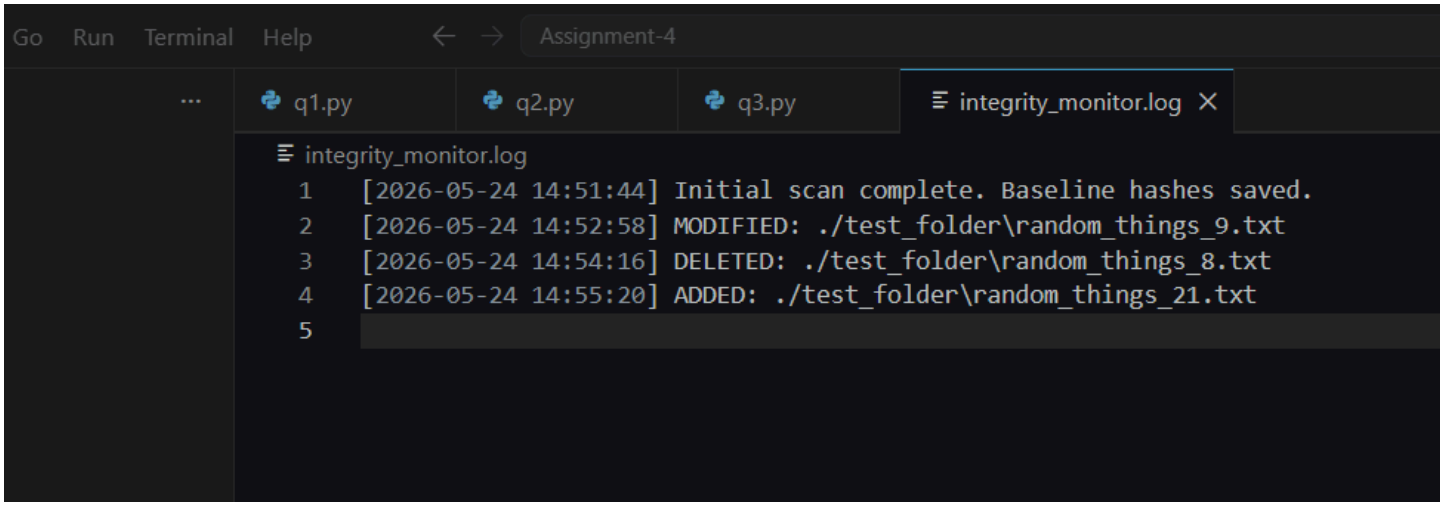
```
PS D:\ITU-SE-SurvivalKit\Semester 6\IS\Assignment\Assignment-4> & C:\Users\Zunaira\AppData\Local\Pro
ester 6/IS/Assignment/Assignment-4/q3.py"
Scanning folder: ./test_folder...
[2026-05-24 14:52:58] MODIFIED: ./test_folder\random_things_9.txt
PS D:\ITU-SE-SurvivalKit\Semester 6\IS\Assignment\Assignment-4>
```

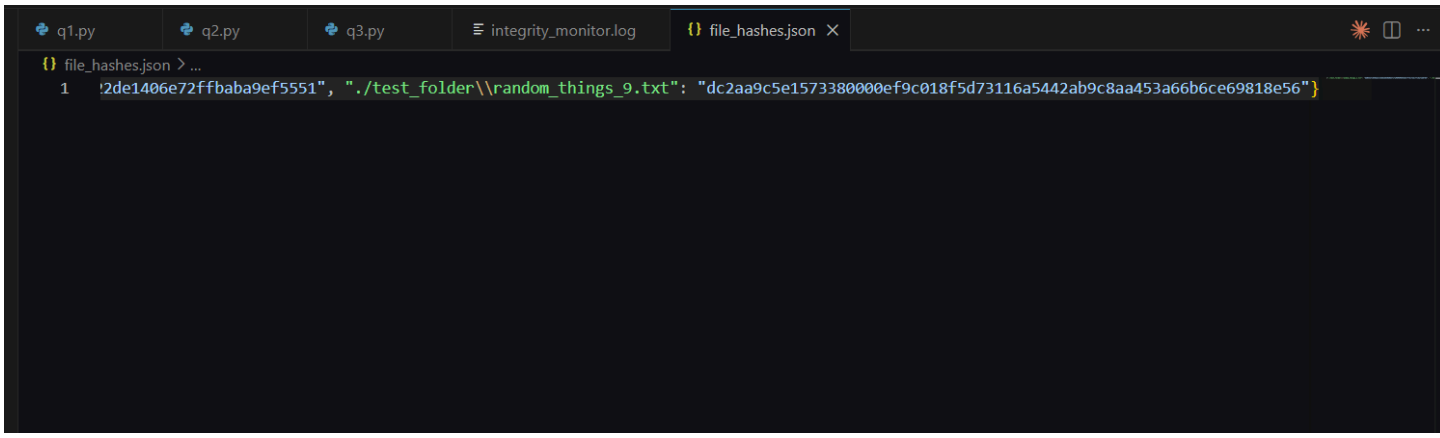
Delete a file:

```
PS D:\ITU-SE-SurvivalKit\Semester 6\IS\Assignment\Assignment-4> & C:\Users\Zunaira\AppData\Local\Programs\Pytho
ester 6/IS/Assignment/Assignment-4/q3.py"
Scanning folder: ./test_folder...
[2026-05-24 14:54:16] DELETED: ./test_folder\random_things_8.txt
PS D:\ITU-SE-SurvivalKit\Semester 6\IS\Assignment\Assignment-4>
```

Adding a brandnew file:

```
PS D:\ITU-SE-SurvivalKit\Semester 6\IS\Assignment\Assignment-4> & C:\Users\Zunaira\AppData\L
ester 6/IS/Assignment/Assignment-4/q3.py"
Scanning folder: ./test_folder...
[2026-05-24 14:55:20] ADDED: ./test_folder\random_things_21.txt
PS D:\ITU-SE-SurvivalKit\Semester 6\IS\Assignment\Assignment-4>
```


Go Run Terminal Help
Assignment-4
q1.py
q2.py
q3.py
integrity_monitor.log
integrity_monitor.log
1 [2026-05-24 14:51:44] Initial scan complete. Baseline hashes saved.
2 [2026-05-24 14:52:58] MODIFIED: ./test_folder\random_things_9.txt
3 [2026-05-24 14:54:16] DELETED: ./test_folder\random_things_8.txt
4 [2026-05-24 14:55:20] ADDED: ./test_folder\random_things_21.txt
5


q1.py
q2.py
q3.py
integrity_monitor.log
file_hashes.json
file_hashes.json > ...
1 2de1406e72ffbaba9ef5551", "./test_folder\\random_things_9.txt": "dc2aa9c5e1573380000ef9c018f5d73116a5442ab9c8aa453a66b6ce69818e56"}